

OOP

Object-Oriented Programming – programming methodology that defines n object rather than functions and logic.

Object – an instance of a class

- Has characteristics or attributes

Example:

```
class Bicycle {  
    // field of class  
    int gear = 5;  
    // method of class  
    void braking() {  
    }  
}  
} // create object
```

Bicycle sportsBicycle = new Bicycle();

State – values of an object's attributes

- First gear

Behaviors – actions that objects can take

- Breaking, accelerating

Method – behavior defined by piece of Java class

- Collection of statements that are grouped together to perform an operation.

Class – blueprint for defining an object

- Object of same kind, same data type = same class
- Data type of obj. = name of its class

Inheritance – one way of organizing classes

- Allows objects of a class to take on properties of objects from another class.
- Extends

Interface – program component that contains the heading for a number of public methods.

- Implements
- Use to specify methods that a class must implement.

Package – collection of related classes and interface

- Name of the folder = name of the package
- import java.util.Scanner;
- import java.util.*;

Void – method is not use to return a value

Parameter – list of variables in a method declaration.

- Dog1 (String n, String b){

Argument – Actual value that is passed

Instance variables – variables declared outside the method.

Local variables – variables declared outside the method.

this – object's name receiving the method call

Information hiding – mechanism for restricting access

- Components are easier to understand/use
- Simplifies modification and repair
- Facilitated re-use

Public Instance Variable – no restriction where you can use its name.

Private instance Variable – name cannot be used to access it outside a class

Public method – can invoke anywhere without restriction

Private method – method cannot be invoked within the definitions of method.

Accessor – returns data from private i.v

Mutator – changes the data stored in 1 or more private i.v

Encapsulation – combining data and actions into a single item

- Groups instance variable and methods into a class
- Hides implementation details

UML Class Diagram – describe the structure of class by displaying the:

1. Class name
2. Variables
3. Methods

Reference – address of the object's memory location

- Class types are reference type
- Instead of ==, use equals()

Constructor – create and initialize an obj.

- Uses new keyword
- MyClass mc = new myClass();
- Does not have a return type
- Can call methods within its class

Static Variables – shared by all objects of its class

- Can be public or private
- Not constants

Static Method – method that can be invoked without using any objects.

- Written in a static keyword
- Write the class name instead of the object name.
- Cannot reference an instance variable of the class.

Predefined Math Class – provides a number of standard mathematical methods.

- Power = Math.pow();
- Absolute value = Math.abs();
- Maximum value = Math.max();
- Minimum value = Math.min();

- Random # = Math.random();
- Rounding = Math.round();
- Ceiling = Math.ceil();
- Floor = Math.floor();
- Square root = Math.sqrt();

Overloading – when multiple methods have the same name within the same class.

- Done by having different method definitions

Signature – Methods name and the number & types of its parameters.

Inheritance - the use of existing class to define a new class

Derived class – adding instance variables and methods to an existing class.

- Subclass
- Does not inherit any constructors from the base class.

Base class – Existing class that the derived class is built upon.

- Superclass

Override – If a derived class defines a method with the same name, the same number and types of parameters and the same return type as a method in the base class.

Super – first action in a constructor definition

Polymorphism – ability of an object to take on many forms.

- Allows to make changes in the method definition.
- Base class reference is used to refer to derived class object.

Dynamic Binding – allows many meanings to be associated to one method name.

- The definition of a method is not bound to an invocation of the method until run time.
- Mechanism used by polymorphism

Abstract class – base for subclasses

Abstract method – declared as abstract and does not have an implementation.

- Written without braces followed by a semicolon.
- public abstract void methodName();

CONSTRUCTORS

```
class Main {
    private String name;

    // constructor
    Main() {
        System.out.println("Constructor Called:");
        name = "Programiz";
    }

    public static void main(String[] args) {

        // constructor is invoked while
        // creating an object of the Main class
        Main obj = new Main();
        System.out.println("The name is " + obj.name);
    }
}
```

THIS KEYWORD

```
class Main {
    int instVar;
    Main(int instVar){
        this.instVar = instVar;
        System.out.println("this reference = " + this);
    }
    public static void main(String[] args) {
        Main obj = new Main(8);
        System.out.println("object reference = " + obj);
    }
}
```

PRIVATE ACCESS MODIFIER

```
class Data {
    // private variable
    private String name;
}

public class Main {
    public static void main(String[] main){
        // create an object of Data
        Data d = new Data();
        // access private variable and field from another
        class
        d.name = "Programiz";
    }
}
```

PUBLIC ACCESS MODIFIER

```
// Animal.java file
// public class
public class Animal {
    // public variable
    public int legCount;
    // public method
    public void display() {
        System.out.println("I am an animal.");
        System.out.println("I have " + legCount + " legs.");
    }
}

// Main.java
public class Main {
    public static void main( String[] args ) {
        // accessing the public class
        Animal animal = new Animal();
        // accessing the public variable
        animal.legCount = 4;
        // accessing the public method
        animal.display();
    }
}
```

ENCAPSULATION

```
class Area {

    // fields to calculate area
    int length;
    int breadth;
    // constructor to initialize values
    Area(int length, int breadth) {
        this.length = length;
        this.breadth = breadth;
    }
    // method to calculate area
    public void getArea() {
        int area = length * breadth;
        System.out.println("Area: " + area);
    }
}

class Main {
    public static void main(String[] args) {
        // create object of Area
        // pass value of length and breadth
        Area rectangle = new Area(5, 6);
```

```
        rectangle.getArea();
    }
}
```

INFORMATION HIDING

```
class Person {
    // private field
    private int age;
    // getter method
    public int getAge() {
        return age;
    }
    // setter method
    public void setAge(int age) {
        this.age = age;
    }
}

class Main {
    public static void main(String[] args) {
        // create an object of Person
        Person p1 = new Person();
        // change age using setter
        p1.setAge(24);
        // access age using getter
        System.out.println("My age is " + p1.getAge());
    }
}
```

INHERITANCE

```
class Animal {
    // field and method of the parent class
    String name;
    public void eat() {
        System.out.println("I can eat");
    }
}

// inherit from Animal
class Dog extends Animal {
    // new method in subclass
    public void display() {
        System.out.println("My name is " + name);
    }
}

class Main {
    public static void main(String[] args) {
        // create an object of the subclass
        Dog labrador = new Dog();
```

```
// access field of superclass
labrador.name = "Rohu";
labrador.display();
// call method of superclass
// using object of subclass
labrador.eat();
}
}
```

METHOD OVERRIDING

```
class Animal {
    public void displayInfo() {
        System.out.println("I am an animal.");
    }
}
class Dog extends Animal {
    @Override
    public void displayInfo() {
        System.out.println("I am a dog.");
    }
}
class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.displayInfo();
    }
}
```

USE OF SUPER KEYWORD

```
class Animal {
    public void displayInfo() {
        System.out.println("I am an animal.");
    }
}
class Dog extends Animal {
    public void displayInfo() {
        super.displayInfo();
        System.out.println("I am a dog.");
    }
}
class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.displayInfo();
    }
}
```

ACCESS SPECIFIER IN OVERRIDING

```
class Animal {
    protected void displayInfo() {
        System.out.println("I am an animal.");
    }
}
class Dog extends Animal {
    public void displayInfo() {
        System.out.println("I am a dog.");
    }
}
class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.displayInfo();
    }
}
```

ABSTRACT CLASS AND METHOD

```
abstract class Animal {
    abstract void makeSound();

    public void eat() {
        System.out.println("I can eat.");
    }
}
class Dog extends Animal {
    // provide implementation of abstract method
    public void makeSound() {
        System.out.println("Bark bark");
    }
}
class Main {
    public static void main(String[] args) {
        // create an object of Dog class
        Dog d1 = new Dog();
        d1.makeSound();
        d1.eat();
    }
}
```

INTERFACE

```
// create an interface
interface Language {
    void getName(String name);
}

// class implements interface
class ProgrammingLanguage implements Language {

    // implementation of abstract method
    public void getName(String name) {
        System.out.println("Programming Language: " +
name);
    }
}

class Main {
    public static void main(String[] args) {
        ProgrammingLanguage language = new
ProgrammingLanguage();
        language.getName("Java");
    }
}
```

POLYMORPHISM

```
class Language {
    public void displayInfo() {
        System.out.println("Common English Language");
    }
}

class Java extends Language {
    @Override
    public void displayInfo() {
        System.out.println("Java Programming Language");
    }
}

class Main {
    public static void main(String[] args) {

        // create an object of Java class
        Java j1 = new Java();
        j1.displayInfo();

        // create an object of Language class
        Language l1 = new Language();
        l1.displayInfo();
    }
}
```

```
}
}
```

INHERITANCE

```
// field and method of the parent class
String name;
public void eat() {
    System.out.println("I can eat");
}

// inherit from Animal
class Dog extends Animal {

    // new method in subclass
    public void display() {
        System.out.println("My name is " + name);
    }
}

class Main {
    public static void main(String[] args) {

        // create an object of the subclass
        Dog labrador = new Dog();

        // access field of superclass
        labrador.name = "Rohu";
        labrador.display();

        // call method of superclass
        // using object of subclass
        labrador.eat();
    }
}
```